

Lecture 2

VLSI Design Flow overview

Prof Peter YK Cheung

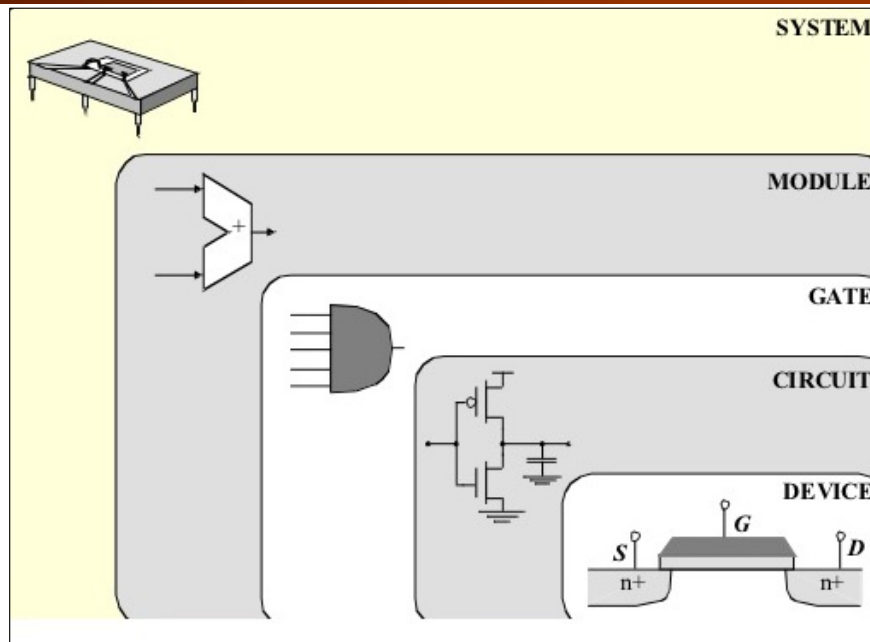
Department of Electrical & Electronic Engineering

URL: www.ee.ic.ac.uk/pcheung/teaching/EE4-VLSI/
E-mail: p.cheung@imperial.ac.uk

In this lecture, we examine in some details how to go from RTL description of a digital circuit all the way to layout. You will learn the various information that are needed to complete this process.

This lecture is closely linked to Lab 1 of this module.

Levels of Abstraction in Digital Design



Digital design, unlike analogue IC design, faces two main challenges: handling complexity involved, and how to verify that such a complex design works correctly. To handle the complexity involved, a designer needs to perform the design task at various level of abstract.

When designing a VLSI chip with millions to billions transistor, a chip architect has to start at the system level (so called System-on-Chip SoC design). The SoC is divided into self-contained modules. For example, in the SoC chip in L1S21, the chip is divided into modules show in rectangle blocks.

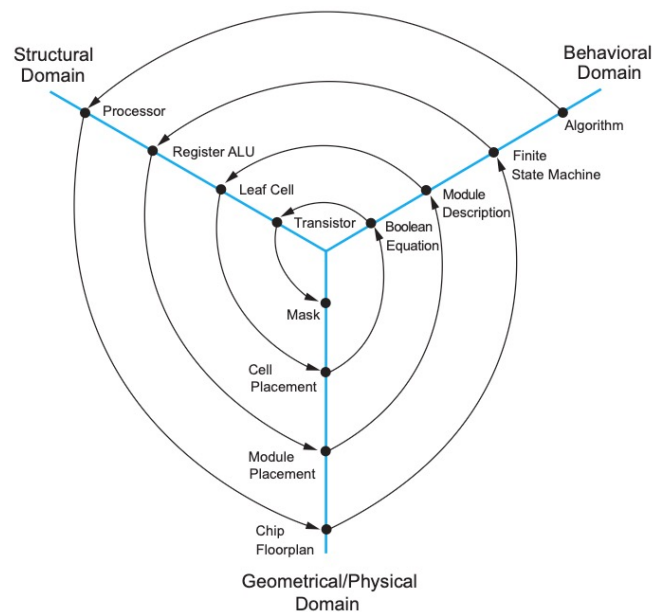
Each module is then converted into smaller digital building block, such as gates or standard cells.

The next level down is the circuit level, where each gate is made up of transistors. In our case, PMOS and NMOS.

Finally each transistor is made up of layers of materials that form the device itself.

For this module, in order that complexity can be handled, we will be designing only down to the gate or standard cell level of abstraction, not lower.

Levels of Abstraction in Digital Design



Source: Gajski 1983 & Kang 2003

This is known as the Y-chart originally introduced by Gajski in 1983. It provides an alternative view on design partitioning using three domains: behavioural, structural and geometrical or physical.

Behavioural domain has the highest level of abstraction. It describes the designer's intent without specifying exactly how the intention is achieved.

The structural domain covers the next few levels from modules down to transistors. It describes the interconnection of gates and module, or even transistors.

The physical domain includes physical locations of gates and cells and placement of cells and blocks and the mask layers.

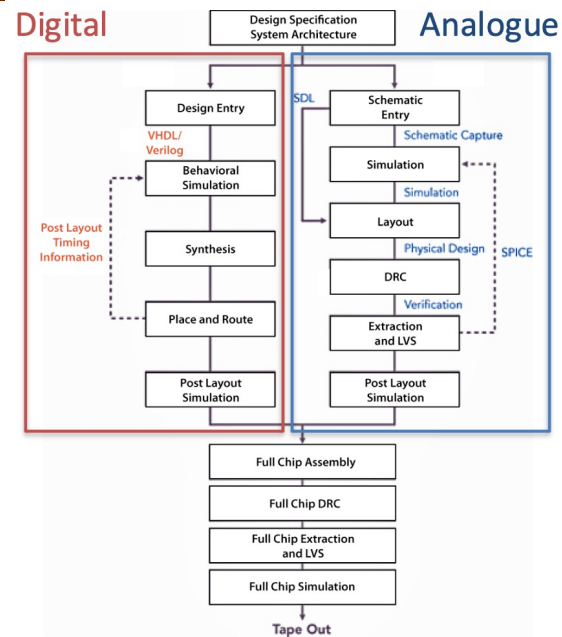
EDA (Electronics Design Automation) tools are used to transform or map descriptions from behavioural to structural, and finally to geometrical domains. Further tools are then used to go up the hierarchy, to verify that the final physical design matches the behavioural description. This is the verification process.

```
graph TD; A[Design Specification] --> B[Behavioral Description]; B --> C[RTL Description (HDL)]; C --> D[Functional Verification and Testing]; D --> E[Logic Synthesis/ Timing Verification]; E --> F[Gate-Level Netlist]; F --> G[Logical Verification and Testing]; G --> H[Floor Planning Automatic Place and Route]; H --> I[Physical Layout]; I --> J[Layout Verification]; J --> K[Implementation]; K --> L[Implementation]; L --> G; J --> C; I --> D; G --> C;
```

The flowchart illustrates the IC design process, starting with Design Specification, followed by Behavioral Description, RTL Description (HDL), Functional Verification and Testing, Logic Synthesis/ Timing Verification, Gate-Level Netlist, Logical Verification and Testing, Floor Planning Automatic Place and Route, Physical Layout, Layout Verification, and Implementation. The process includes feedback loops from Logical Verification and Testing back to RTL Description (HDL), from Layout Verification back to Physical Layout, and from Implementation back to Logical Verification and Testing.

Postlayout simulation. Check to see the design still works with the added loads of the interconnect.

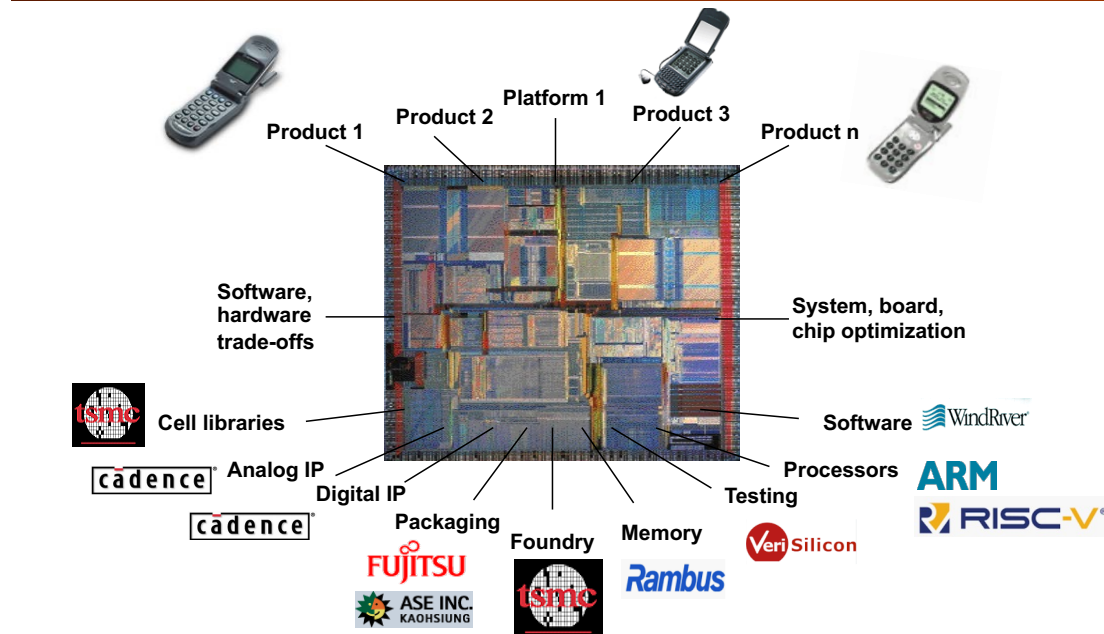
Mixed signal VLSI Design Flow



Source: Notes from T Constantinou

This is another view of how a high-level specification of a design is mapped to physical layout. With mixed signal design, much effort is put into the circuit design and simulation stage, particular for analogue part of the chip.

IP based VLSI Design

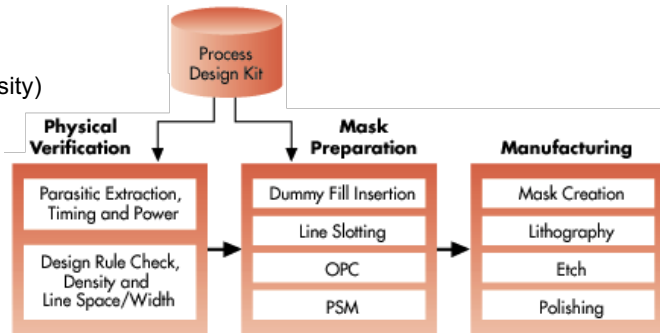


Another approach to handle the complexity of modern VLSI chips is to build a chip using intellectual property (IP) blocks designed by others. Here is an example where different companies provide the various IP modules that are fully tested down to the layout.

This approach has many advantages. It reduces the need for in house expertise in a large variety of subjects. It further reduces both the time for designing a complex VLSI chip and reduces the risk involved.

What is a Process Design Kit (PDK)

- ◆ Link between chip maker (foundry) and EDA tool supplier (TSMC → Cadence)
- ◆ PDK is for a specific process (we use TSMC 65nm low power process)
- ◆ PDK consists for set of files in standardised format for different design steps:
 - Primitive devices (e.g. transistors, capacitor etc.)
 - Device parameters
 - Device symbol
 - Technology related information
 - Layers and layer names
 - Appearance (colour)
 - Constraints (e.g. current density)
 - Verification information
 - Design rule checking (DRC)
 - Layout vs Schematic (LVS)
 - Device extraction
 - Simulation models
 - Standard cell libraries
 - Documentation



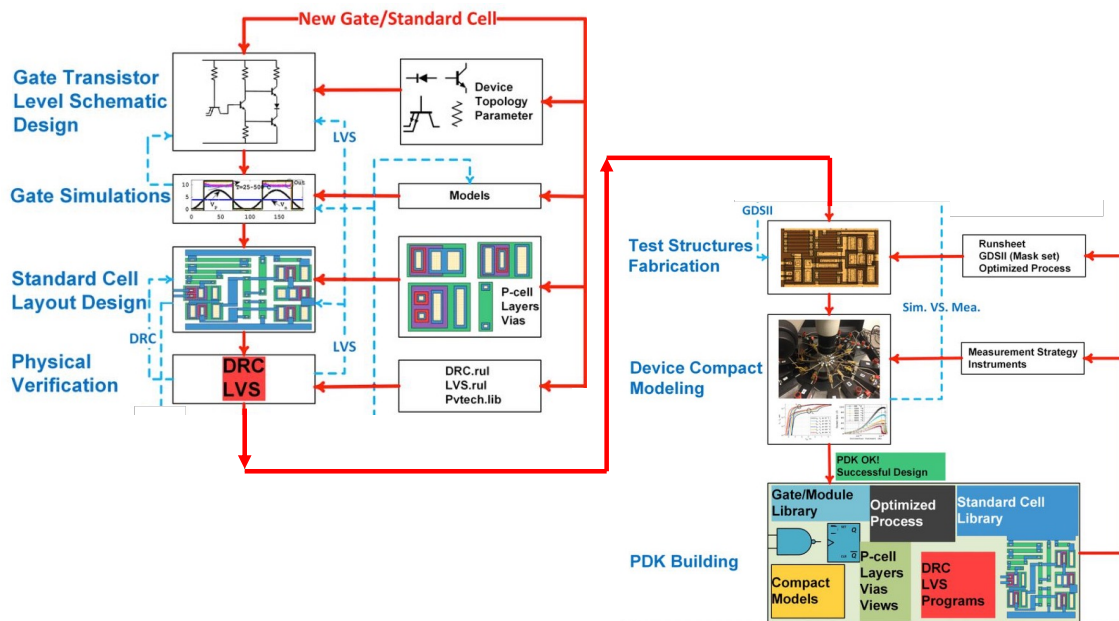
Process Design Kit (PDK) is provided by the Foundry or chip maker for specific EDA tool platform. In our case, we use TSMC 65nm PDK for Cadence. (Every student must sign a non-disclosure agreement before accessing the PDF.)

The PDK consists of a many files divided into different categories, and includes a set of fully tested and characterised standard cells.

The files can be roughly divided into the categories shown above. These are used for:

- Full-custom design using individual **primitive devices** such as PMOS and CMOS transistors, capacitors, diodes etc.
- Information relating to the **process technology** such as different mask layers, what colour they appears in the EDA tools and some properties such as current density in metal etc.
- Information necessary for **verifying the chip** works properly. These include design rules (e.g. minimum width of a layer, minimum separation between layouts etc.); extraction rules – how layers together form a device or a component.
- **Simulation model** – for analogue (SPICE-like) and digital simulators.
- **Standard cell library** – facilitate technology mapping, timing optimization and place-and-route optimization.
- **Documentation** and design guidelines.

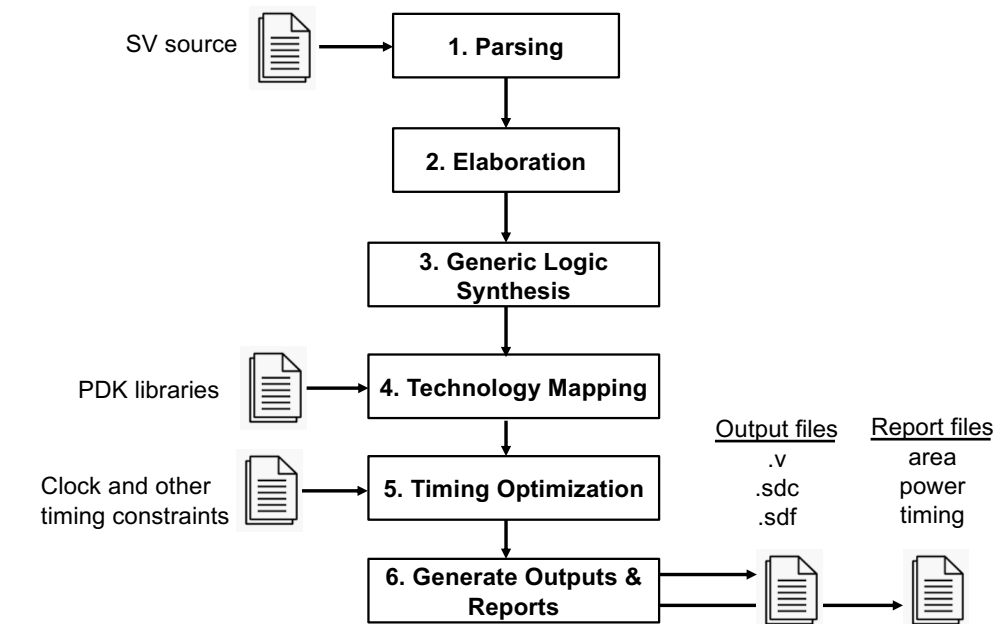
PDK Build Process



Source: Gupta

This slide explains how one would build your own PDK or your own cell library. However, we are not going to do this in our module due to the time it takes and the risk involved.

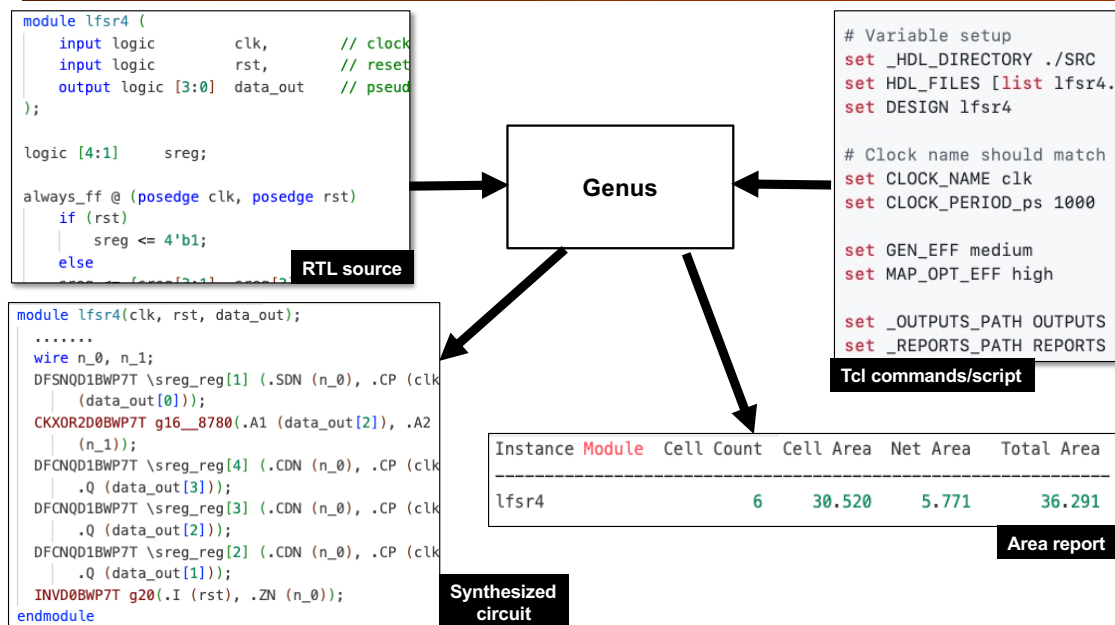
Genus - RTL to Standard Cell Synthesis



Genus is Cadence's tool for RTL to **standard cell synthesis**. This tool has a number of steps which will be explored during Lab 1.

1. **Parsing** – Check for syntax and other basic errors in SystemVerilog sources.
2. **Elaboration** – Expand modules and submodules, resolves parameters, process generate statements and check that all signals are connected..
3. **Generic logic synthesis** - Partition RTL specification into control and dataflow graphs (CDFG) and perform initial synthesis. Turn the nodes of graphs into generic gates (e.g. NAND3 or DFF etc.) no matter what the process technology is to be used. The output of this process is a netlist of generic gates.
4. **Technology mapping** – Map generic gates to TSMC standard cells. It also choose the correct variant of cell based on the timing, area and power constraints. It also insert tie cells (e.g. tie-hi or tie-lo) and clock gating cells as needed.
5. **Timing optimization** – Analyse timing and use the timing constraints (e.g. clock period), optimize the standard cells for the required timing requirements (i.e. setup and hold times). Also choose the drive strength of cells for the required performance. Buffers may be inserted.
6. **Output and report generation** – Write synthesised outputs including netlist in form of Verilog sources (.v), timing constraints (.sdc) and delay information (.sdf). Also write reports on important parameters such as area, power and delay.

Synthesize LFSR4



Genus takes RTL source code (SystemVerilog in our case). It also needs lots of Genus specific commands. The command syntax is that of Tcl (pronounced as tickle). You can enter these commands one at a time or source them from a script file (.tcl).

The result of Genus is a synthesized netlist in Verilog format (not SystemVerilog). It also produces timing constraints file, delay file and various reports (e.g. area, timing, power etc.).

Synthesis Steps (preparation) (1)

```
# Variable setup
set _HDL_DIRECTORY ./SRC
set HDL_FILES [list lfsr4.
set DESIGN lfsr4

# Clock name should match
set CLOCK_NAME clk
set CLOCK_PERIOD_ps 1000

set GEN_EFF medium
set MAP_OPT_EFF

# Tell Genus about the clock signal
create_clock -domain domain1 -name ${CLOCK_NAME}
set_db clock:${DESIGN}/${CLOCK_NAME} .setup_unc
set_clock_uncertainty -hold [expr 0.02 * ${CLOCK_PERIOD}]
set_clock_uncertainty -setup [expr 0.02 * ${CLOCK_PERIOD}]
set_clock_transition -rise 50 ${CLOCK_NAME}
set_clock_transition -fall 50 ${CLOCK_NAME}

# Load the design and check basic syntax
read_hdl -sv ${HDL_FILES}
elaborate ${DESIGN}
check_design -unresolved
check_design -unloaded
```

Generally Genus is driven through a Tcl script (in our case, synth.tcl).

Running Genus involved many distinct steps. Before synthesis can start, the script needs to:

- ❖ Set environment variable to say what files to use, folders containing files, name of clock signals, and other parameters.
- ❖ Set up database about standard cells and other useful information for the process using the PDK provided.
- ❖ Perform elaboration step and basic syntax checking.
- ❖ Inform Genus about the clock signal and various clock constraints so that timing optimisation can be performed during synthesis.

Synthesis Steps (synthesis) (2)

```
# First synthesize to generic gates
set_db syn_generic_effort $GEN_EFF
syn_generic

# Then map to TSMC standard cells
set_db syn_map_effort $MAP_OPT_EFF

# Optimize netlist

# Export Design Files
write_snapshot -directory ${_REPORTS_PATH}/final -tag final
report_summary -directory ${_REPORTS_PATH}
write_hdl > ${_REPORTS_PATH}/hdl.rpt
write_sdc > ${_REPORTS_PATH}/sdc.rpt
write_sdf > ${_REPORTS_PATH}/sdf.rpt
write_script > ${_REPORTS_PATH}/script.rpt
write_design > ${_REPORTS_PATH}/design.rpt
write_db -all ${_REPORTS_PATH}/db

# Write reports
report_qor > ${_REPORTS_PATH}/${_DESIGN}_qor.rpt
report_area > ${_REPORTS_PATH}/${_DESIGN}_area.rpt
report_dp > ${_REPORTS_PATH}/${_DESIGN}_datapath_incr.rpt
report_messages > ${_REPORTS_PATH}/${_DESIGN}_messages.rpt
report_gates > ${_REPORTS_PATH}/${_DESIGN}_gates.rpt
report_timing > ${_REPORTS_PATH}/${_DESIGN}_timing.rpt
report_power > ${_REPORTS_PATH}/${_DESIGN}_power.rpt
```

The next stage of the process are:

- ❖ Synthesizes the RTL to generic gates, not caring at this stage about the process technology.
- ❖ Map to cells in TSMC 65nm standard cells library.
- ❖ Optimize the resulting circuits according to specified timing constraints.
- ❖ Export various output files including the synthesized netlist.
- ❖ Write report files for user with detailed information.

Genus synthesized circuit

```
module lfsr4 (  
    input logic clk, // clock  
    input logic rst, // reset  
    output logic [3:0] data_out // pseud  
);
```

```
    logic [4:1] sreg;  
  
    always_ff @ (posedge clk, posedge  
        if (rst)  
            sreg <= 4'b1;  
        else  
            sreg <= {sreg[3:1], sreg[3] ^  
    assign data_out = sreg;  
endmodule
```

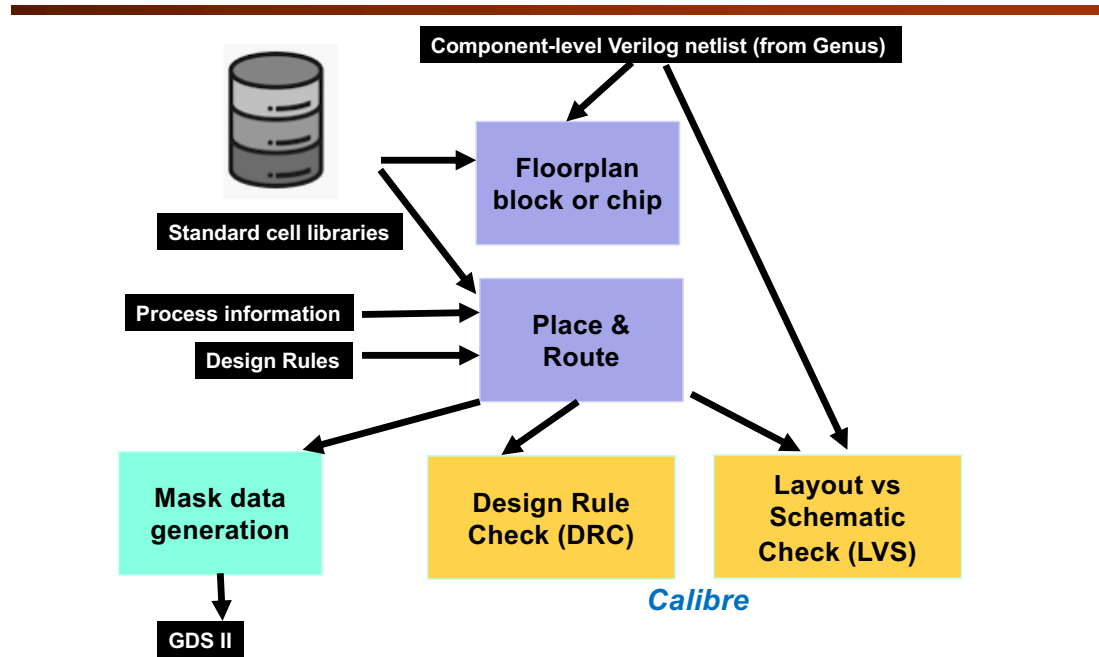
**RTL source
in SystemVerilog**

```
module lfsr4(clk, rst, data_out);  
    input clk, rst;  
    output [3:0] data_out;  
    wire clk, rst;  
    wire [3:0] data_out;  
    wire n_0, n_1;  
    DFSNQD1BWP7T \sreg_reg[1] (.SDN (n_0), .CP (clk), .D (n_1), .Q  
        (data_out[0]));  
    CKXOR2D0BWP7T g16__8780(.A1 (data_out[2]), .A2 (data_out[3]), .Z  
        (n_1));  
    DFCNQD1BWP7T \sreg_reg[4] (.CDN (n_0), .CP (clk), .D (data_out[2]),  
        .Q (data_out[3]));  
    DFCNQD1BWP7T \sreg_reg[3] (.CDN (n_0), .CP (clk), .D (data_out[1]),  
        .Q (data_out[2]));  
    DFCNQD1BWP7T \sreg_reg[2] (.CDN (n_0), .CP (clk), .D (data_out[0]),  
        .Q (data_out[1]));  
    INVDOBWP7T g20(.I (rst), .ZN (n_0));  
endmodule
```

**synthesized netlist
in Verilog**

Here is how Genus turns the lfsr4.sv source code into a netlist of connected standard cells. As can be seen here, the synthesized circuit has four D-FF (two variants: DFSNQD1BWP7T and DFCNQD1BWP7T), an XOR gate (CKXOR2D0BWP7T) and an inverter (INVDOBWP7T).

Innovus in context



After synthesis, the tool to use to produce the actual layout is the placement and routing tools (PnR).

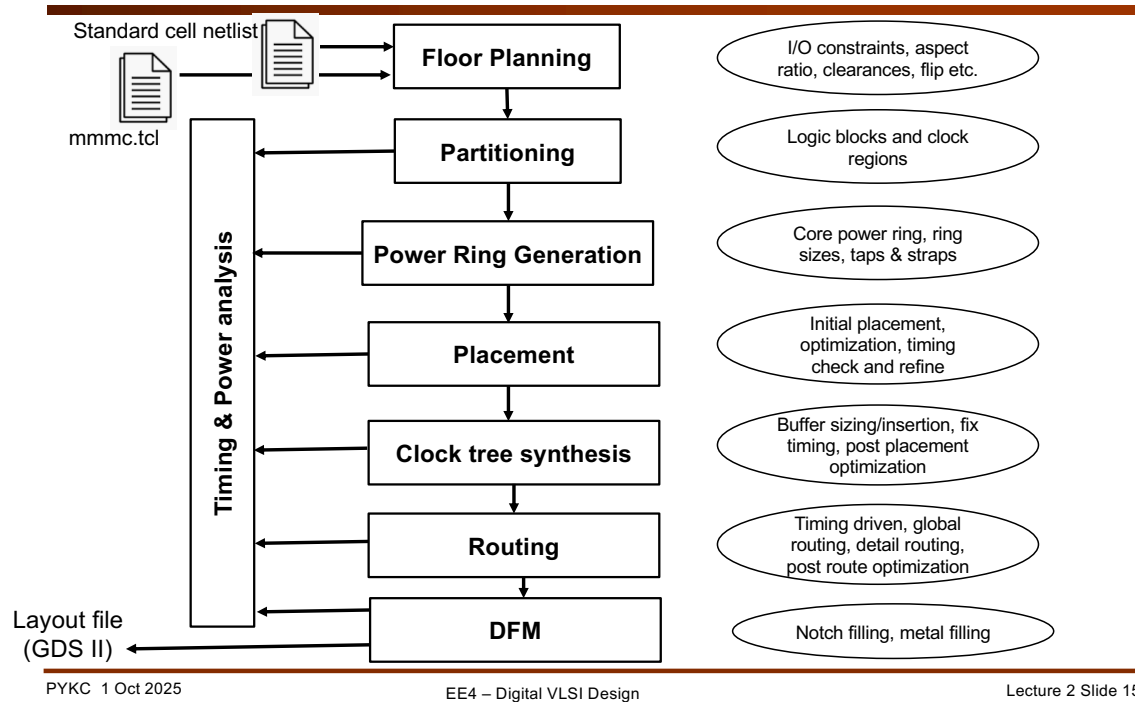
Innovus is Cadence's PnR tool. It takes synthesized netlist and standard cell library from the PDK, perform floorplanning (rough placement with global routing) before placing the cells on the chip and conduct detail routing. Various timing-driven optimization steps are conducted before a final design is achieved.

The resulting layout is checked for design rule violation (Design Rule Checker DRC) and verified to be identical to the schematic (i.e. synthesized netlist) using LVS. The place-and-routed design is also verified through simulation.

Innovus performs basic DRC and LVS. However detail DRC and LVS are done through another set of tools known as Calibre. This will be covered later in the course.

Finally, once everything are verified, the synthesized and place-and-routed design is used to generate mask data set in GDS II format, and ship to the foundry. That is known as the tapeout process.

Innovus – Netlist to Layout Flow



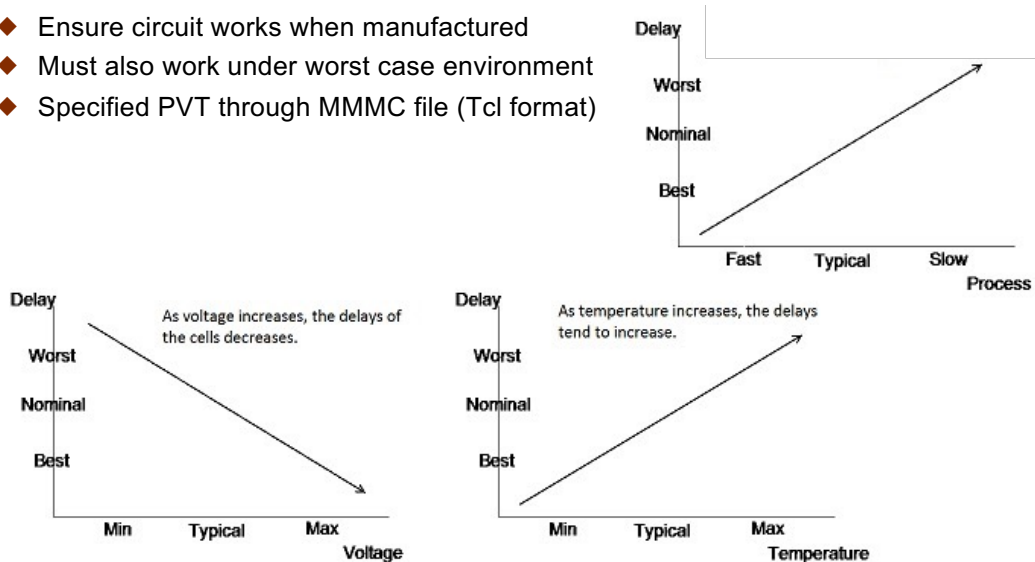
This slide shows the details steps that Innovus go through and what each step achieves.

Note that Innovus requires additional information supplied by the file “mmmc.tcl”. This will be explained later.

Further, note that at each stage of PnR, Innovus performs timing analysis to modify the layout to optimize the timing – a process known as timing-driven PnR.

What is PVT?

- ◆ PVT stands for Process, Voltage and Temperature
- ◆ Need to tell Place-and-Route tools these variations
- ◆ Ensure circuit works when manufactured
- ◆ Must also work under worst case environment
- ◆ Specified PVT through MMMC file (Tcl format)



MMMC file is a specification file that inform PnR process about variations that the chip will experience. These variations include process (P), supply voltages (V) and temperature (T), collectively known as PVT.

The three graphs above depict how each of these three factors affects the gate delays. These variation information is provided in the PDK so that the PnR tool will create a layout that meets the timing specification under the worst case condition (called corners).

PYKC 1 Oct 2025
EE4 – Digital VLSI Design
Lecture 2 Slide 17

Innovus, just like Genus, uses Tcl syntax for its commands. These can be entered interactively or sourced from a Tcl script storing all the commands in a file.

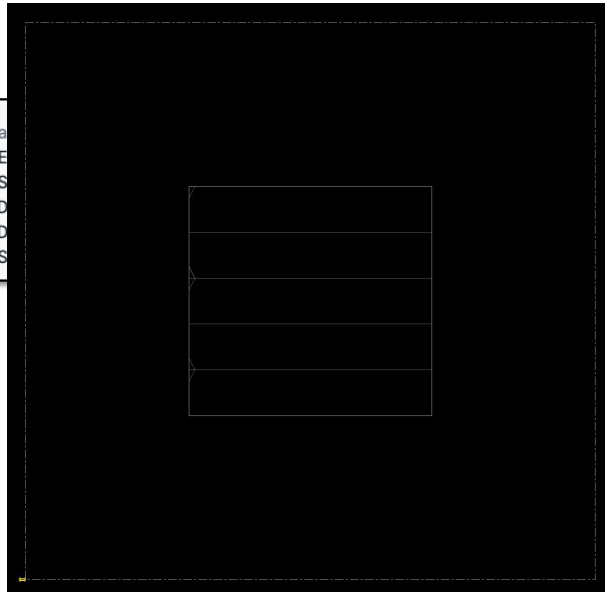
PnR steps – Initial Floor Planning

```
#Top-level module name
set DESIGN lfsr4
set VERILOG_FILES [list OUTPUTS/lfsr4_synth.v]
set SDC_FILE OUTPUTS/lfsr4_synth.sdc

# S
set # Plan how the overall layout, particularly
set floorplan -r ${CORE_ASPECT_RATIO} ${CORE
set
set globalNetConnect VSS -type pgpin -pin VS
globalNetConnect VDD -type pgpin -pin VD
# S
# T
globalNetConnect VDD -type tiehi -pin VD
set globalNetConnect VSS -type tielo -pin VS
set
set POWER_RING_OFFSET 0.5

# Setup PnR environment and say where library c
set init_verilog ${VERILOG_FILES}
set init_top_cell ${DESIGN}
set init_lef_file { \
  /usr/local/cadence/kits/tsmc/beLibs/65nm/TS
  /usr/local/cadence/kits/tsmc/beLibs/65nm/TS
  /usr/local/cadence/kits/tsmc/beLibs/65nm/TS
}
set init_mmmc_file "./mmmc_timing.tcl"
set init_gnd_net {VSS}
set init_pwr_net {VDD}

setDesignMode -process 65
init_design
```



Here are the Tcl script segments that drives Innovus to perform PnR. The first group of Tcl commands set up the environment variables and specify locations of files etc.

It then perform an initial design which define the bounding box of the layout.

This is followed by a preliminary floorplan. Here, the layout aspect ratio is specified to be 1 (or square), therefore the preliminary floorplan has the standards cells staging on top of each other as shown.

PnR steps – Power ring, supply routing & well taps

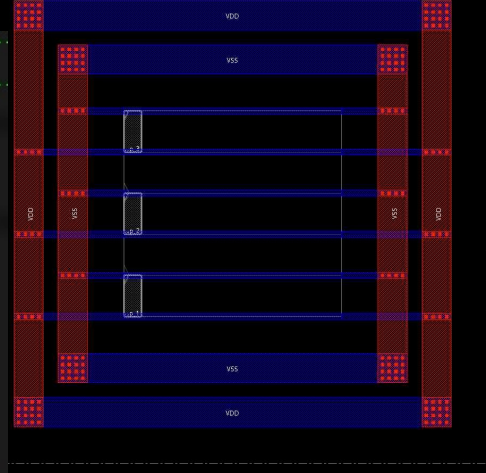
```
# Add power ring
addRing -width ${POWER_RING_WIDTH} -spacing $0
```

timeDesign Summary

Setup views included:
functional_typ

Setup mode	all	reg2reg	default
WNS (ns):	0.384	0.384	0.871
TNS (ns):	0.000	0.000	0.000
Violating Paths:	0	0	0
All Paths:	8	4	4

Density: 61.932%



```
prePlace -prefix prePlace
```

The next few steps are:

- ❖ Add the power ring – VDD and GND.
- ❖ Route the power to each row of the standard cells – the horizontal metal.
- ❖ Add well taps to prevent latch up – we will deal with latch up late in the course.
- ❖ Perform timing analysis to estimate how fast this will run.

The reported timing of this circuit

WNS = worst negative slack

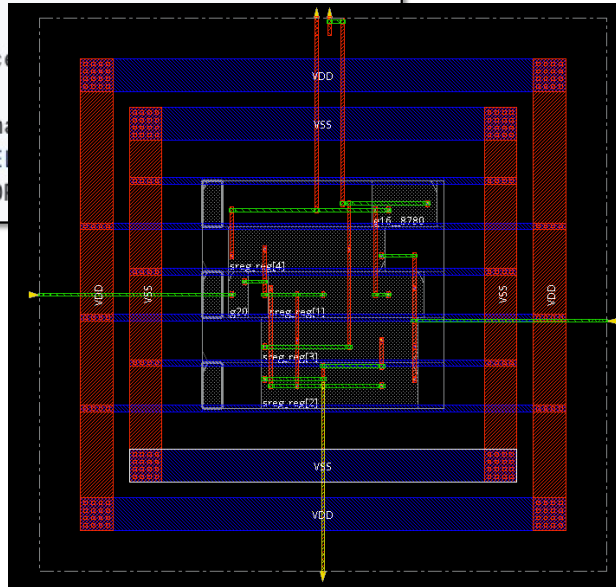
TNS = total negative slack

The table above shows that WNS is 0.384ns – this is the most pessimistic slack time (i.e. timing margin = required time arrival time – actual arrival time). In other words, we have 384ps spare of the 1ns clock period we specified. The minimum clock period under all worst-case PVT condition is 616ps!

This is of course only an estimate. We have not finished PnR yet.

PnR steps - Placement

```
# Placement
setPlaceMode -place_global_place
place_opt_design
setTieHiLoMode -maxFanout 10 -ma
addTieHiLo -cell "TIEHBWP7T TIE
timeDesign -preCTS -outDir REPO
```



The next step is detail placement – putting each standard cell in a specific location on the chip. This step also perform preliminary routing to provide better estimate of the timing.

The layout now has all the cells and some routing in place.

PnR steps – Clock Tree Synthesis (CTS)

```
# specify clock properties
set_ccopt_property buffer_cells {CKBD0BWP7T CKBD1BWP7T CKBD2BWP7T CKBD3BWP7T}
set_ccopt_property inverter_cells {CKND0BWP7T CKND1BWP7T CKND2BWP7T CKND3BWP7T}
set_ccopt_property target_max_trans 130ps
set_ccopt_property target_skew 200ps
set_ccopt_property max_fanout 20

setOptMode -usefulSkew true
setOptMode -usefulSkewCCOpt extreme

# clock tree synthesis (CTS)
create_ccopt_clock_tree_spec -file REPORTS/ctsspec.tcl
source REPORTS/ctsspec.tcl
ccopt_design -check_prerequisites
ccopt_design

# post clock tree synthesis optimization
optDesign -postCTS -setup -hold -outDir REPORTS/postCTSOptTiming
timeDesign -postCTS -expandedViews -outDir REPORTS/postCTS -prefix postCTS
report_ccopt_clock_trees -file REPORTS/postCTS/clock_trees.rpt
report_ccopt_skew_groups -file REPORTS/postCTS/skew_groups.rpt
```

The next step is very important – that of synthesizing the layout of the clock tree circuit. This step ensures that the clock circuit is optimized so that the clock skew and clock related timing constraints are satisfied.

PnR steps – Routing & Finish up

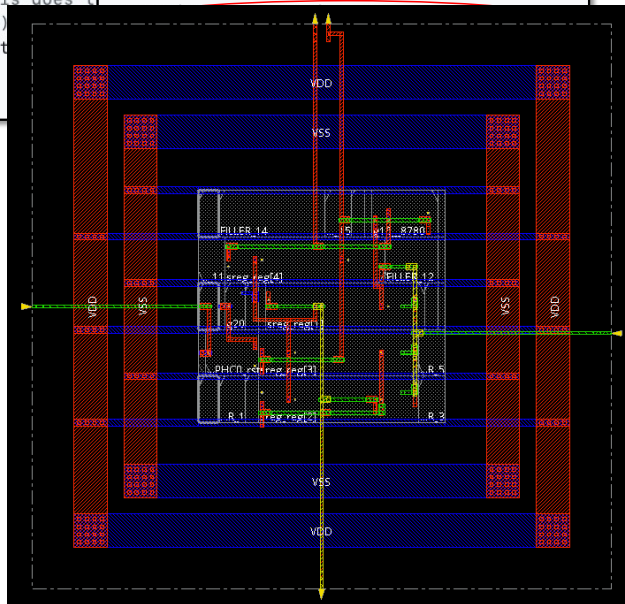
```
routeDesign                                ;# this does t
## Set analysis on on-chip variation (OCV)
setAnalysisMode -analysisType onChipVariat
optDesign -postRoute -setup
optDesign -postRoute -hold
```

lfsr4_soc.v

```
module lfsr4 (
    clk,
    rst,
    data_out);
    input clk;
    input rst;
    output [3:0] data_out;

    // Internal wires
    wire FE_PHN0_rst;
    wire n_0;
    wire n_1;

    DEL01BWP7T FE_PHN0_rst (.I(rst),
        .Z(FE_PHN0_rst));
    DFNQD1BWP7T \sreg_reg[1] (.CP(clk),
        .D(n_1),
        .Q(data_out[0]),
        .SDN(n_0));
    CKX0R208BWP7T g16_8780 (.A1(data_out[2]),
        .A2(data_out[3]),
        .Z(n_1));
    DFNQD1BWP7T \sreg_reg[4] (.CDN(n_0),
        .CP(clk),
        .D(data_out[2]),
        .Q(data_out[3]));
    DFNQD1BWP7T \sreg_reg[3] (.CDN(n_0),
        .CP(clk),
        .D(data_out[1]),
        .Q(data_out[2]));
    DFNQD1BWP7T \sreg_reg[2] (.CDN(n_0),
        .CP(clk),
        .D(data_out[0]),
        .Q(data_out[1]));
    INV08BWP7T g20 (.I(FE_PHN0_rst),
        .ZN(n_0));
endmodule
```



Finally, we perform detail routing and optimization. This step complete the PnR process. It includes further optimization of setup and hold time or insert buffers as necessary.

It then add filler cells to eliminate gaps between standard cells. As a result, the final layout has standard cells forming continuous rows without gaps.

It then perform a basic Design Rule Check (DRC) and Layout to Schematic (LVS) verification.

Final step is to output files and reports.

Testbench (lfsr4_tb.v)

```

timescale 1ns/1ps
module lfsr4_tb;
    logic rst;
    logic clk;
    logic [3:0] data_out;

    // instantiate the DUT
    lfsr4 dut (
        .clk(clk),
        .rst(rst),
        .data_out(data_out)
    );

    initial begin
        $display("Start lfsr4 testbench ....");
        clk = 1'b0;
        rst = 1'b1;
        #1 clk = 1'b1;
        #1 clk = 1'b0;
        #1 rst = 1'b0;

        // clock generator
        for (int i=0; i<20; i++) begin
            #1 clk = 1'b1;
            #1 clk = 1'b0;
        end
    end

    initial begin
        $dumpfile("waveform.vcd");
        $dumpvars(1, lfsr4_tb, dut);
    end
endmodule

```

Makefile

```

# Directory definitions
LIB_DIR = "/usr/local/cadence"
SRC_DIR = ./
SDF_DIR = ./
WAVE_DIR = ./waves

# File definitions
TESTBENCH = SRC/lfsr4_tb.v
DUT_SRCS = OUTPUTS/lfsr4_soc

# Run simulation with SDF and
run:
    ncverilog $(LIB_DIR)
    ncverilog -sv $(DUT_SRCS)

# View waveforms (assuming .
waves:
    gtkwave waveform.vcd &

```

Xcelium (ncverilog)

GTKwave

Signals

Time	clk	rst	data_out[3:0]
1	0	1	0000
2	1	1	0000
3	0	1	0000
4	1	1	0000
5	0	1	0000
6	1	1	0000
7	0	1	0000
8	1	1	0000
9	0	1	0000
10	1	1	0000
11	0	1	0000
12	1	1	0000
13	0	1	0000
14	1	1	0000
15	0	1	0000
16	1	1	0000
17	0	1	0000
18	1	1	0000
19	0	1	0000
20	1	1	0000

Waves

10 ns 20 ns 30 ns 40 ns

1 2 4 9 3 6 10 A 5 B 7 F E C 8 1 2 4 9 3 6

Before simulation can be done, one has to create a testbench design for the circuit. The output is a waveform file (waveform.vcd) which can be viewed with GTKwave.

Further Reading

- ◆ Publicly available standard cell library:
https://www.vlsitechnology.org/html/cells/vsclib013/lib_gif_index.html
- ◆ Genus Synthesis Flow Guide, v18.1 – stored on teaching servers
/usr/local/cadence/GENUS_18.10.000/doc/genus_start/genus_start.pdf
- ◆ Innovus User Guide, v18.1 – stored on teaching servers
/usr/local/cadence/INNOVUS_18.10.000/doc/innovusUG/innovusUG.pdf
- ◆ “New VLSI Tools”, Gajski&Kuhn ([10.1109/MC.1983.1654264](https://doi.org/10.1109/MC.1983.1654264))

TSMC standard cell library is under NDA and therefore its details cannot be disclosed. However, list in the slide is an excellent open-source standard cell library including most of the commonly used gates and circuit snippets.

Cadence manuals are not publicly available, but are shipped with the software installation under a “doc” folder. Here are the two main users guides for Genus and Innovus. Be warned – these are really very long. The Innovus User Guide has 1,595 pages! Happy reading?